University of Exeter

ECM2433: The C Family

Lecture 03: Ownership Fundamentals

Why Ownership?

- C/C++ give you control, but memory bugs are common
- Use-after-free, double-free, memory leaks
- C++ smart pointers help but aren't enforced
- Rust's ownership: compiler-checked memory safety, no garbage collector

University of Exeter

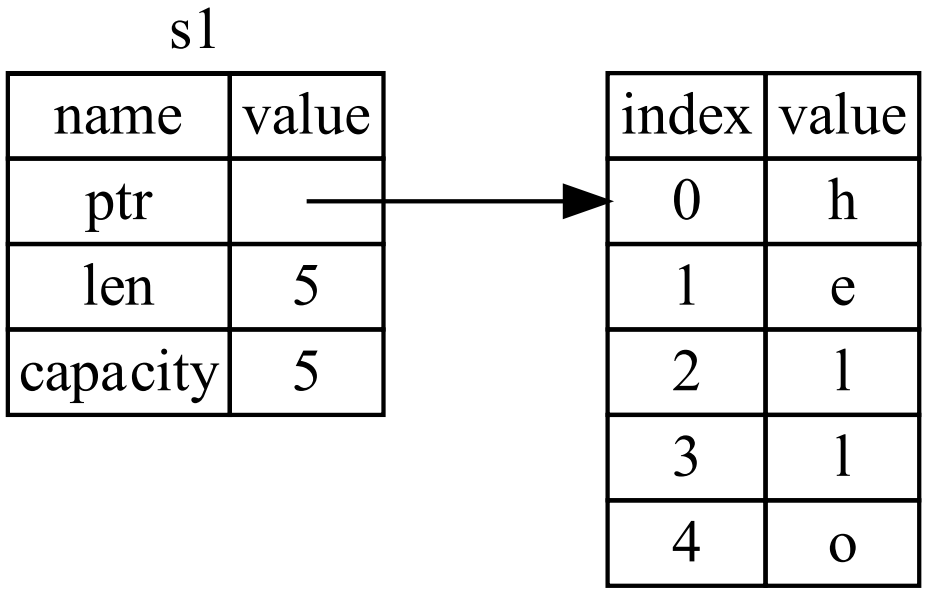
Stack vs Heap

- Stack: fast, fixed-size data (integers, bools, chars)
- Heap: dynamic data, accessed via pointer (strings, vectors)
- Heap data needs clean-up: who frees the memory?
- Ownership rules answer this question at compile time

The `String` Type

```
1 let s1 = String::from("hello");
```

- `String::from`: creates a heap-allocated string
- Stack holds pointer, length, and capacity
- Heap holds the actual character data
- We'll cover `String` in detail next video



University of Exeter

The Three Ownership Rules

```
1 {
2     let s = String::from("hello");  // s owns the String
3     // use s here
4 }   // s goes out of scope, String is dropped (memory freed)
```

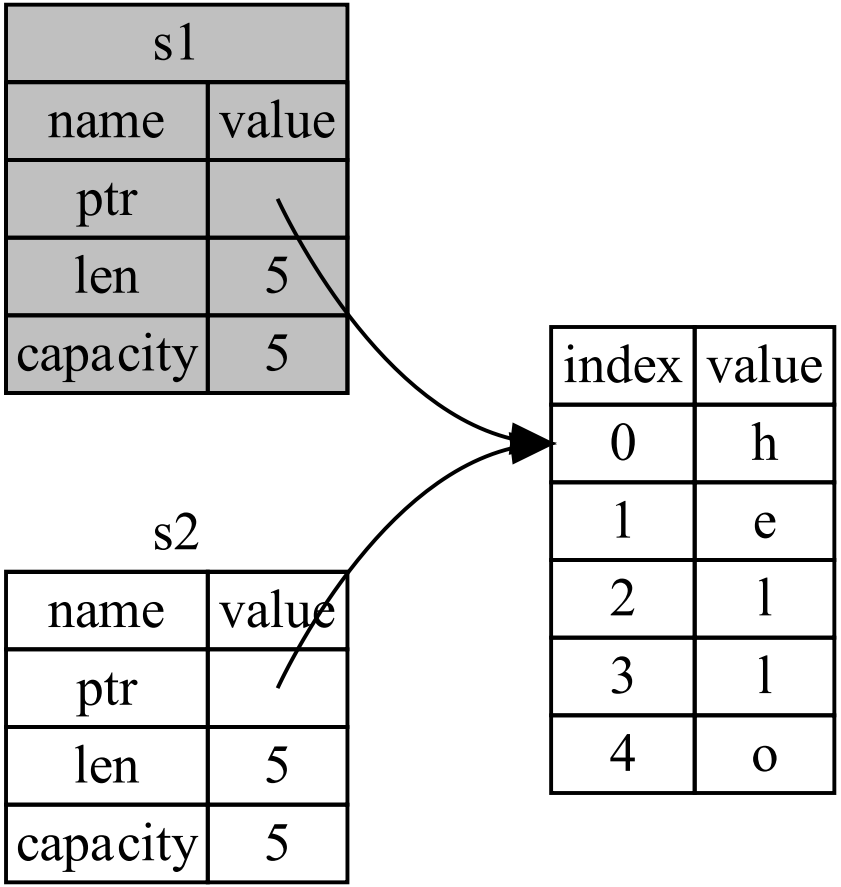
- 1. Each value in Rust has an owner
- 2. There can only be one owner at a time
- 3. When the owner goes out of scope, the value is dropped

University of Exeter

Move Semantics

```rust
let s1 = String::from("hello");
let s2 = s1;          // s1 is moved to s2

// works
println!("{s2}");
// error: value borrowed after move
println!("{s1}");
```



- The original variable becomes invalid
- Compiler prevents use-after-move

University of Exeter

`Copy` Types and `clone()`

```rust
let x = 5;
let y = x;          // x is copied, not moved

println!("{x}");    // works
println!("{y}");    // works
```

```rust
let s1 = String::from("hello");
let s2 = s1.clone(); // explicit copy

println!("{s1}");   // works
println!("{s2}");   // works
```

- Simple scalar types implement `Copy`: integers, floats, `bool`, `char`
- Tuples implement `Copy` if all elements do
- `Copy` types are duplicated on assignment; both variables remain valid
- For heap types, use `.clone()` to explicitly copy the data

University of Exeter

References and Borrowing

```rust
fn takes_ownership(s: String) {
    println!("{s}");
}   // s is dropped here

fn borrows(s: &String) {
    println!("{s}");
}   // reference goes out of scope, nothing dropped

fn main() {
    let s1 = String::from("hello");
    takes_ownership(s1);
    println!("{s1}");     // error: s1 was moved

    let s2 = String::from("world");
    borrows(&s2);
    println!("{s2}");     // works: s2 still valid
}
```

- Passing a non-`Copy` value moves ownership; the caller loses access
- `&`: creates a reference, borrows without taking ownership
- When a reference goes out of scope, the data is not dropped

University of Exeter

Mutable References

```rust
fn add_world(s: &mut String) {
    s.push_str(", world");
}

fn main() {
    let mut s = String::from("hello");
    add_world(&mut s);
    println!("{s}");  // "hello, world"
}
```

- `&mut`: creates a mutable reference, allows modification
- The owner must be declared `mut`
- Pass with `&mut` to allow the function to modify

University of Exeter

The Borrowing Rules

```rust
let mut s = String::from("hello");

let r1 = &s;
let r2 = &s;
// multiple immutable refs: ok
println!("{r1}, {r2}");

// mutable ref: ok (r1, r2 no longer used)
let r3 = &mut s;
println!("{r3}");
```

- The rule: one `&mut` OR any number of `&` (not both)
- Multiple `&`: allowed (everyone is reading)
- Multiple `&mut`: not allowed (only one writer)
- Mix of `&` and `&mut`: not allowed while `&` still in use

```rust
let mut s = String::from("hello");

let r1 = &mut s;
// error: cannot borrow as mutable more than once
let r2 = &mut s;
```

Dangling References

```rust
fn dangle() -> &String {
    let s = String::from("hello");
    &s  // error: s will be dropped, reference would be invalid
}

fn no_dangle() -> String {
    let s = String::from("hello");
    s   // return owned value instead
}
```

- The rule: references must always be valid
- Cannot return a reference to a local variable
- Fix: return the owned value instead

University of Exeter

Recap

- Ownership: each value has one owner; dropped when owner goes out of scope
- Moves: heap types transfer ownership on assignment; original becomes invalid
- `Copy`: simple scalar types are duplicated; both remain valid
- References: `&T` borrows immutably, `&mut T` borrows mutably
- Borrowing rules: one `&mut` XOR many `&`; references must be valid
- Required reading:
  - Book Chapter 4.1: What Is Ownership?
  - Book Chapter 4.2: References and Borrowing

University of Exeter